

Smart Pointers — Ինչո՞ւ են պետք

C++ հարցազրույցի ամբողջական ուղեցույց

Այս ուղեցույցը բացատրում է, INCHU-ն smart pointer-ները թե՛ raw pointer-ի problem-ներ, RAII, երեք տեսակներ, Rule of Zero/Three/Five, օրինակներով և interview հարցերով:

1. Raw Pointer-ի problem-ներ

Smart pointer-ները լուծում են raw pointer-ի 4 մեք problem.

1.1 Memory Leak

```
void f() {  
    int* p = new int(5);  
    if (error) return;    // LEAK -- delete chi hasni  
    delete p;  
}
```

1.2 Dangling Pointer

```
int* p = new int(5);  
delete p;  
cout << *p;    // DANGLING -- jnjvac memory (UB)
```

1.3 Double Delete

```
int* p = new int(5);  
delete p;  
delete p;    // DOUBLE DELETE -- UB / crash
```

1.4 Ownership anpaշutyun

```
void process(int* p);    // օվ է patasxanatu delete-ի? anhayt
```

Raw pointer-ic haytni chi. ov e owner-y, ov petq e delete ani -> sxaler.

2. Lucumy — Smart Pointers (RAII)

Smart pointer-y object e, vor **avtomat karavarum e memory-n** (RAII) `scope verjum inqn e delete anum.

```
#include <memory>
void f() {
    auto p = std::make_unique<int>(5);
    if (error) return;    // CHKA leak -- avtomat delete
}                        // scope verj -> avtomat delete
```

raw pointer: dzerqov delete, leak/dangling/double-delete vtang
smart pointer: avtomat delete (RAII), ownership parz, chka leak

3. Erek tesaknery

unique_ptr -> EXCLUSIVE ownership (mek owner), zero overhead, move-only
shared_ptr -> SHARED ownership (mi qani owner), reference counting
weak_ptr -> NON-OWNING (chi achecnum count), circular reference-i lucum

unique_ptr

```
auto u = std::make_unique<int>(5);    // mek owner
// copy chka, miayn move
```

shared_ptr

```
auto s1 = std::make_shared<int>(5);   // count 1
auto s2 = s1;                          // count 2 (shared)
```

weak_ptr

```
std::weak_ptr<int> w = s1;           // chi achecnum count -- circular xusapum
```

4. Vory meky erb

```
unique_ptr  -> DEFAULT. mek owner, exclusive resource (90% depqerum)
shared_ptr  -> iskapas mi qani owner petq e (shared data)
weak_ptr    -> shared_ptr-i het, circular reference kotrelu / observer
```

Kanon. **sksir unique_ptr-ic**; anci shared_ptr miayn erb iskapas shared ownership petq e.

5. Rule of Zero / Three / Five

Rule of Three (hin C++)

Ete class-y petq uni destructor, copy constructor, kam copy assignment-ic **meky** (sovorabar raw pointer-i patcharov), petq uni **erek-y** el.

```
class Bad {
    int* data;
public:
    Bad() { data = new int[100]; }
    ~Bad() { delete[] data; }           // 1
    Bad(const Bad&);                    // 2 (deep copy)
    Bad& operator=(const Bad&);        // 3
};
```

Rule of Five (C++11)

- move constructor + move assignment (5 total).

Rule of Zero (LAVAGUYN)

Ete ogtagorcum es smart pointer / STL container -> **OCHMI** dzerqov special member petq che.

```
class Good {
    std::unique_ptr<int[]> data;    // RAII inqny
    std::vector<int> v;
    std::string s;
};    // ochmi dtor/copy/move petq che -- compiler default-nery chisht en
```

6. Inchu smart pointers — amֆօֆ

- + Avtomat memory management (RAII) -> chka leak
- + Chka dangling / double-delete
- + Pařz ownership (unique = mek, shared = qani)
- + Exception-safe (scope verj -> avtomat delete)
- + Rule of Zero -> qich kod, qich sxal
- shared_ptr overhead (atomic count) -> ogtagorci պետք եղաՅս

7. Interview harcer & patasxanner

Inchu smart pointers?

Raw pointer-i problem-neri (leak, dangling, double-delete, anpařz ownership) lucum RAII-ov -> avtomat memory management.

Erek tesaknery?

unique_ptr (exclusive), shared_ptr (shared, ref count), weak_ptr (non-owning, circular lucum).

Vory meky default?

unique_ptr -- zero overhead, exclusive ownership. shared miayn erb iskapēs shared petq e.

Rule of Zero inch e?

RAII tiper (smart pointer/container) ogtagorcelis ochmi dzerqov dtor/copy/move petq che.

Rule of Three/Five?

Three (dtor/copy-ctor/copy-assign) hin; Five (+move ctor/assign) C++11.

shared_ptr overhead?

Atomic reference count + control block -> unique-ic canr; ogtagorci miayn erb shared petq e.

Circular reference inchpes es lucum?

weak_ptr (chi achecnum count) mek uxxutyamb.

Cheat Sheet

RAW POINTER PROBLEMS: leak | dangling | double-delete | anpařz ownership

SMART POINTERS (RAII -> avtomat delete):

unique_ptr -> exclusive (mek owner), zero overhead, DEFAULT

shared_ptr -> shared (mi qani), reference counting

weak_ptr -> non-owning, circular reference lucum

RULE OF:

Three -> dtor + copy-ctor + copy-assign (hin)

Five -> + move-ctor + move-assign (C++11)

Zero -> RAII tiper -> OCHMI dzeřqov (LAVAGUYN)

KANON: sksir unique_ptr-ic; shared miayn erb iskares shared ownership petq e